



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algoritmusok és Adatszerkezetek I.

Pusztai Gábor

Algoritmusok és Alkalmazásaik Tanszék
Informatikai Kar
ELTE - Eötvös Loránd Tudományegyetem, Budapest

2026. április 20.

Tartalom I

- 1 Kupac (Heap)
 - Fa struktúrák
 - Kupac (Heap)
 - Beszúrás
 - Maximum elem törlése
- 2 Prioritásos sor
 - Beszúrás
 - Maximum elem törlés
 - Elemzés
 - Feladat
- 3 Kupacrendezés
 - Kupacrendezés
 - Feladat
 - Elemzése

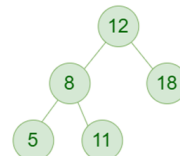
- Hatékonyabban

Kupac (Heap)

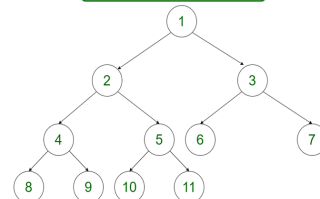
Fa struktúrák

- **Szigorúan bináris fa:** (Strictly, full)
Minden belső pontjának két gyereke van.
- **Majdnem teljes** (Nearly complete)
Melynek legalsó (levél) szintjéről elhagytunk néhány levelet (de nem az összeset).
- **Teljes (majdnem teljes + balra tömörített):**
(Complete, vagy szintfolytonos fa)
Melynek legalsó (levél) szintjéről elhagytunk néhány levelet (de nem az összeset).
Az alsó szintjén a levelek balról-jobbra hézagmentesen helyezkednek el.
- **Tökéletes:** (Perfect)
Minden belső pontjának két gyereke van.
Minden előforduló szintjének teljes a kitöltöttsége.
Azaz olyan szigorúan bináris fa, ahol minden levél azonos szinten helyezkedik el.

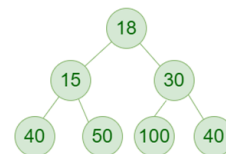
Full Binary Tree



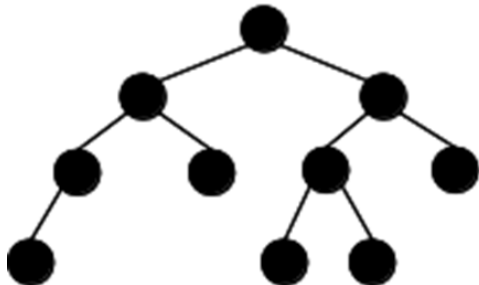
Complete Binary Tree



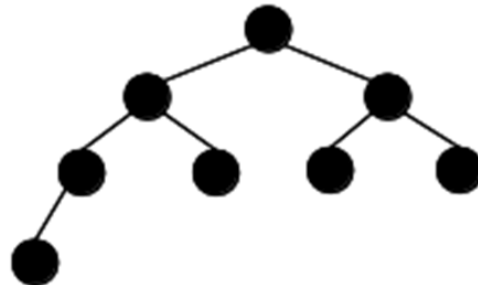
Perfect Binary Tree



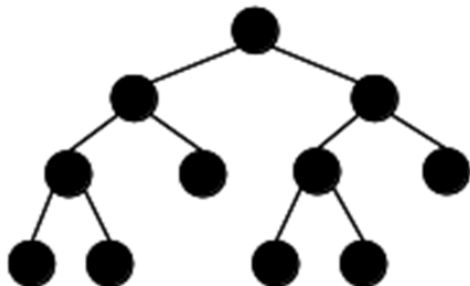
Neither complete nor full



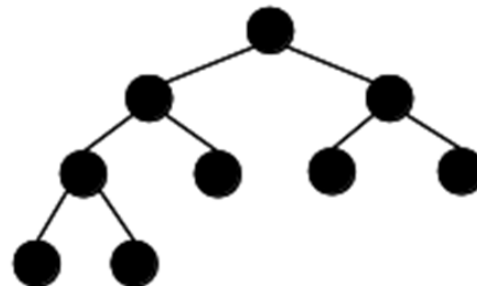
Complete but not full



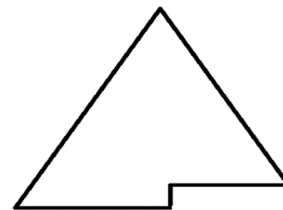
Full but not complete



Complete and full

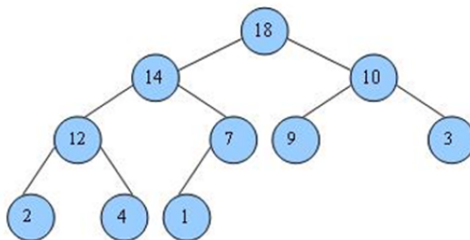


- **Kupac definíciója:** Egy majdnem teljes, balra tömörített bináris fa.
- **Maximum kupac:** minden belső pont kulcsa nagyobb vagy egyenlő a gyerekei kulcsánál. A gyökérben a legnagyobb elem van.
- **Minimum kupac:** a szülő kulcsa kisebb vagy egyenlő a gyerekei kulcsánál.



A maximum kupac definíciója

- A bináris **maximum kupac** egy olyan teljes bináris fa, amelyben minden csúcs kulcsa nagyobb vagy egyenlő ($\geq$), mint a gyerekeinek kulcsa.
- Lehetnek azonos kulcsértékek a kupacban.
- A maximum elem közvetlenül elérhető: ez a gyökér.
- (A **minimum kupac** analóg módon definiálható ($\leq$).)
- Egy n csúcsú teljes bináris fa magassága $\lfloor \log_2 n \rfloor$.

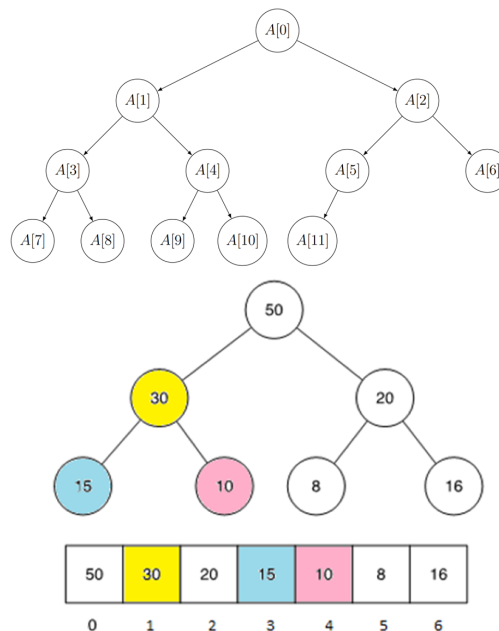


Aritmetikai reprezentáció

A kupacok (és teljes bináris fák) hatékonyan ábrázolhatók tömbbel, mivel a szintfolytonos bejárás egyértelmű és hézagmentes felsorolást ad.

A tömb indexelése 0-tól indul ($A[0..n-1]$):

- Adott csúcs indexe: i
- Bal gyerek indexe: $2i + 1$
- Jobb gyerek indexe: $2i + 2$
- Szülő indexe: $\lfloor \frac{i-1}{2} \rfloor$



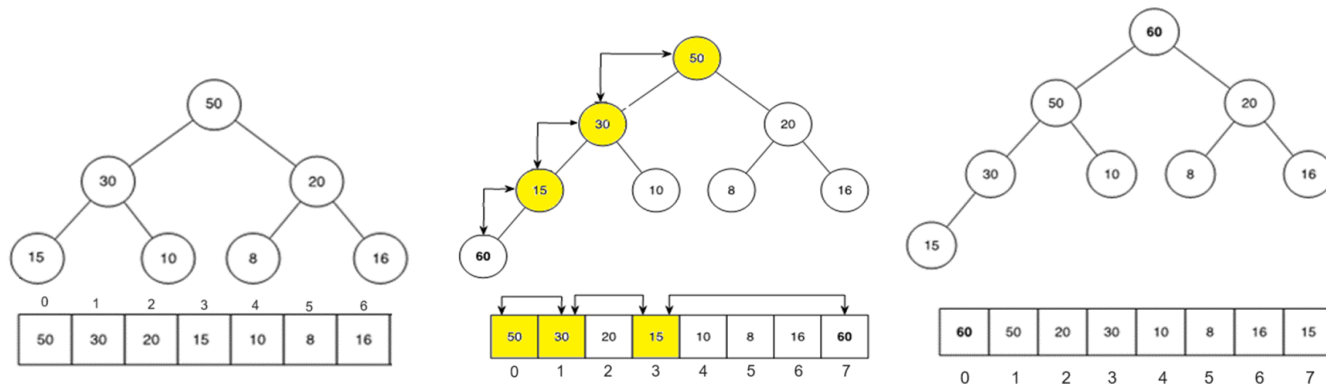
- **Két fő művelet:**
 - Beszúrás (**add**)
 - Maximum kivétele (**remMax**)
- A kupacot aritmetikai reprezentációban adjuk meg:
 $A : T[n]$.
- A tömb végén maradhatnak üres helyek, de „lyukak” az elemek között nem lehetnek.

Beszúrás (Insert)

Példa: Szúrjuk be a 60-at a (50, 30, 20, 15, 10, 8, 16) kupacba!

- 1 Első lépés:** a 60-at a tömb legvégére (az első szabad helyre) tesszük.
- 2 Második lépés:** az új elem „felemelkedik” (*rise*), amíg a kupac-tulajdonság helyre nem áll (cserék a szülővel).

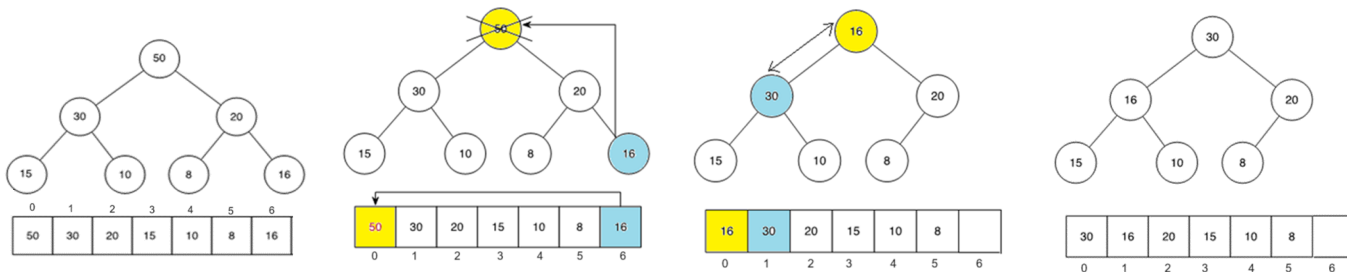
Ez legfeljebb $\lfloor \log_2 n \rfloor$ cserét igényel.



Maximum elem törlése

- 1 A maximum (gyökér) eltávolítása után a tömb utolsó elemét a gyökérbe helyezzük. Ez megőrzi a tömb folytonosságát.
- 2 Ezután az elem „lesüllyed” (*sink*): amíg kisebb bármelyik gyerekenél, felcseréljük a nagyobbik gyerekekkel.

Cserék maximális száma: $\lfloor \log_2 n \rfloor$.



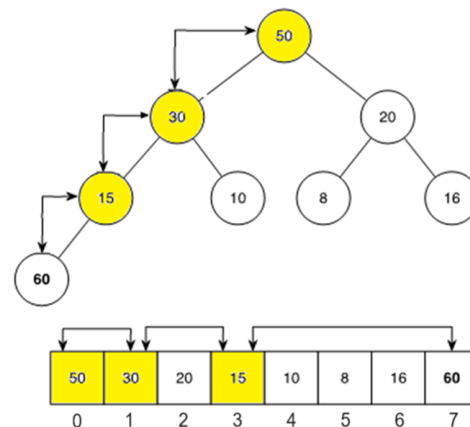
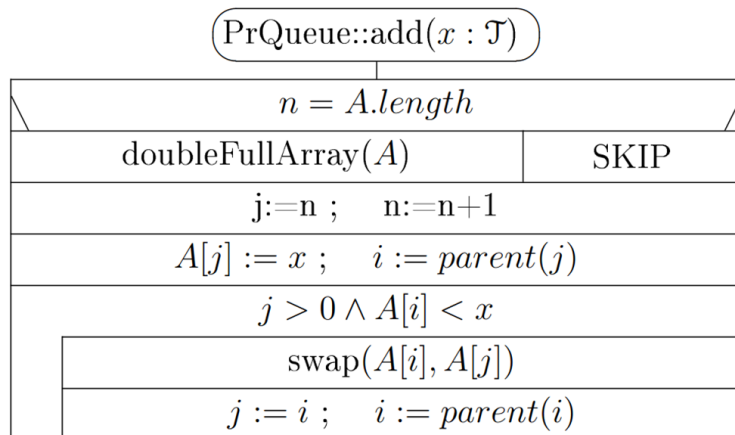
Prioritásos sor

Prioritásos sor (Priority Queue)

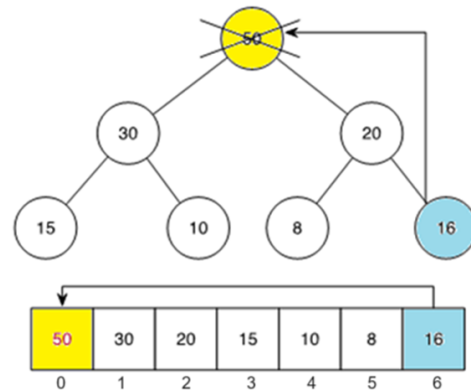
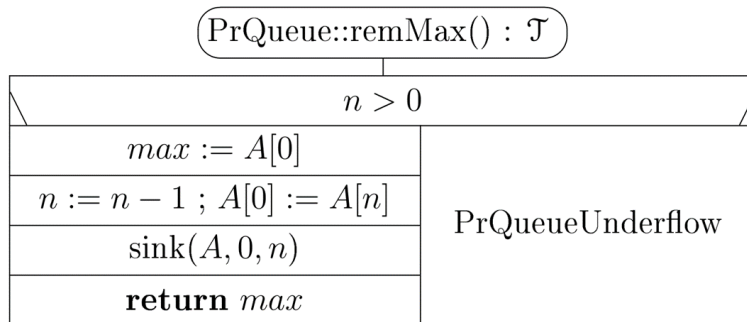
- Olyan sor, amelyből mindig a legnagyobb (vagy legkisebb) elemet vehetjük ki.
- Hatékonyan megvalósítható kupaccal.

PrQueue
- $A/1 : \mathcal{T}[]$ // $\mathcal{T}$ is some known type
- $n : \mathbb{N}$ // $n \in [0..A.length]$ is the actual length of the priority queue
+ PrQueue($m : \mathbb{N}$) { $A := \text{new } \mathcal{T}[m]; n := 0$ } // create an empty priority queue
+ add($x : \mathcal{T}$) // insert x into the priority queue
+ remMax(): $\mathcal{T}$ // remove and return the maximal element of the priority queue
+ max(): $\mathcal{T}$ // return the maximal element of the priority queue
+ isFull() : $\mathbb{B}$ {return $n = A.length$ }
+ isEmpty() : $\mathbb{B}$ {return $n = 0$ }
+ $\sim$ PrQueue() { delete A }
+ setEmpty() { $n := 0$ } // reinitialize the priority queue

Prioritásos sor - beszúrás



Prioritásos sor - maximum elem törlés



Prioritásos sor - süllyesztés

$\text{sink}(A : \mathcal{T}[] ; k, n : \mathbb{N})$

$i := k ; j := \text{left}(k) ; b := \text{true}$

$j < n \wedge b$

// $A[j]$ is the left child of $A[i]$

$j + 1 < n \wedge A[j + 1] > A[j]$

$j := j + 1$

SKIP

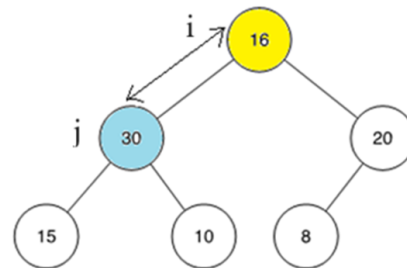
// $A[j]$ is the greater child of $A[i]$

$A[i] < A[j]$

$\text{swap}(A[i], A[j])$

$i := j ; j := \text{left}(j)$

$b := \text{false}$



- Sink(A,1,6)
- Legyen a csúcs indexe: i
- Bal gyerekének indexe: $2 * i$
- Jobb gyerekének indexe: $2 * i + 1$
- Szülő indexe: $\lfloor \frac{i}{2} \rfloor$ ($i/2$ alsó egész része)

Maximum prioritásos sor megvalósításainak összehasonlítása

	<code>add(x)</code>	<code>remMax()</code>	<code>max()</code>
rendezetlen tömb (max indexe)	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$
rendezett tömb (csökkenő)	$O(n)$	$\Theta(1)$	$\Theta(1)$
kupac	$O(\log n)$	$O(\log n)$	$\Theta(1)$

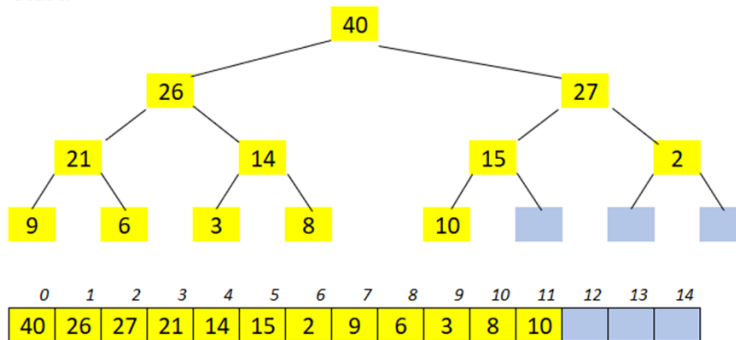
Feladat: Kupac

Adott az alábbi prioritásos sor:

[40, 26, 27, 21, 14, 15, 2, 9, 6, 3, 8, 10, _, _, _].

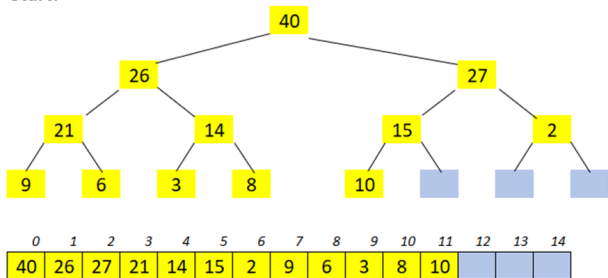
- Mutass be az **add(61)** és az azt követő **add(43)** műveleteket!
- Mutass be két **remMax()** műveletet az eredeti kupacon!
- A folyamatot szemléltesse fa és tömb formában is!

Start:

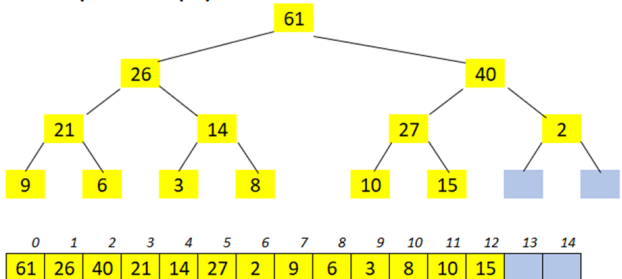


Megoldás: Kupac (Q.add(61))

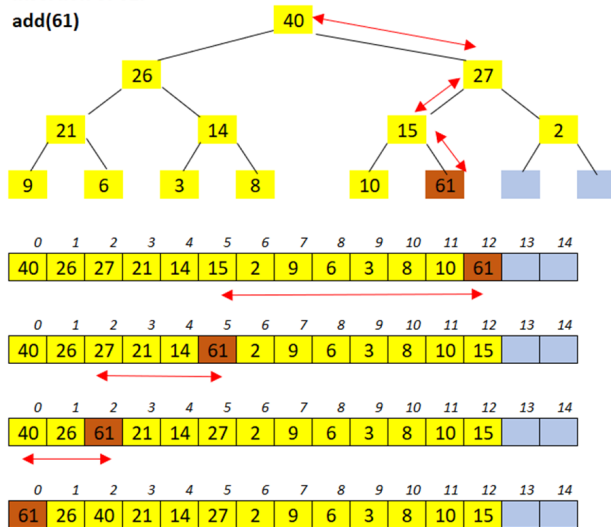
Start:



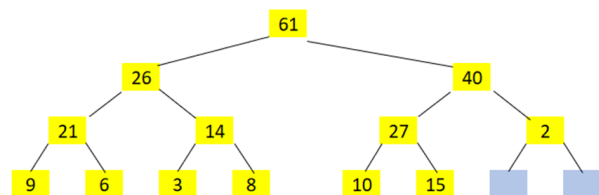
The heap after add(61):



Insertion of 61:
add(61)

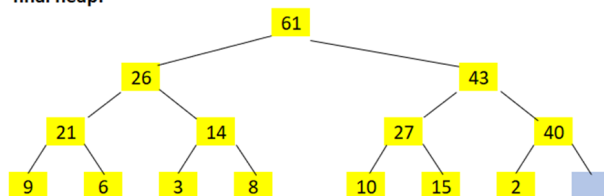


Megoldás: Kupac (Q.add(43))



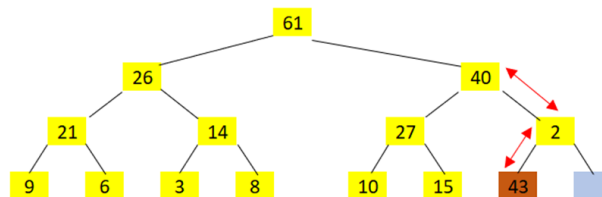
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
61	26	40	21	14	27	2	9	6	3	8	10	15		

final heap:



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
61	26	43	21	14	27	40	9	6	3	8	10	15	2	

add(43):

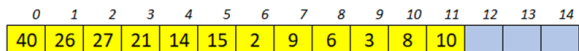
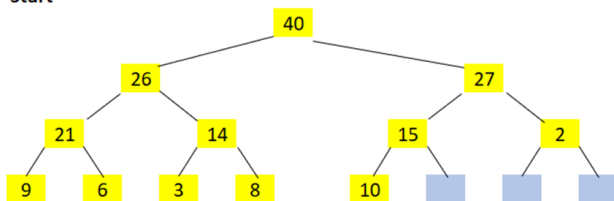


0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
61	26	40	21	14	27	2	9	6	3	8	10	15	43	

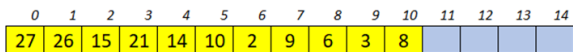
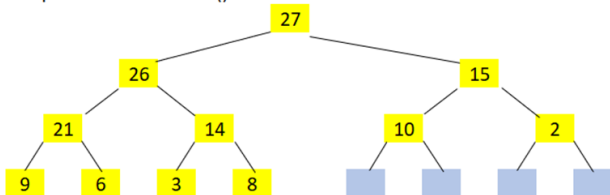
Red arrows indicate the path from index 13 to index 12 and then to index 11, showing the bubble-up process.

Megoldás: Max eltávolítása (Első Q.remMax():)

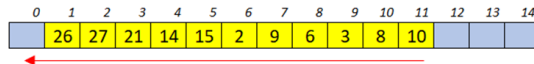
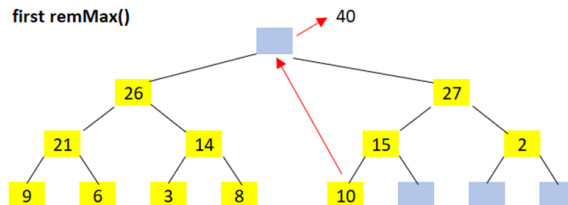
Start



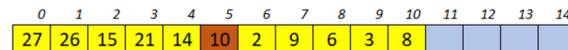
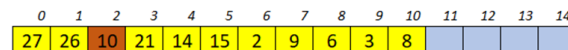
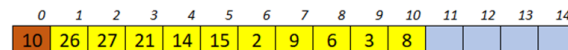
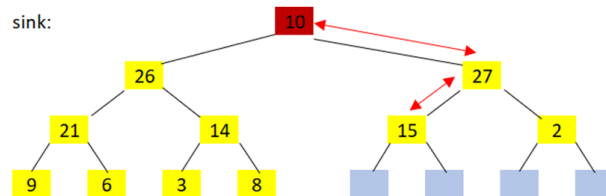
heap after first remMax():



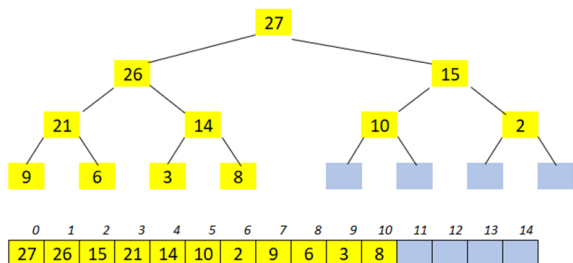
first remMax()



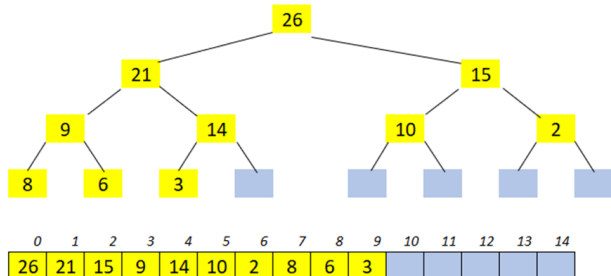
sink:



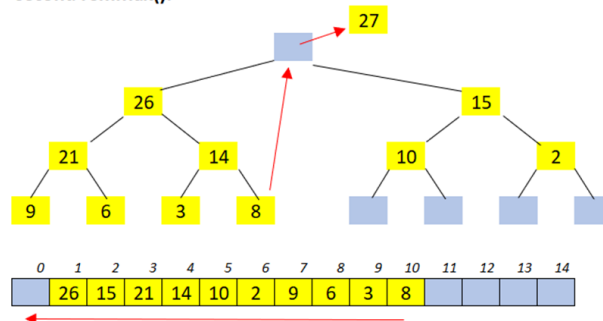
Megoldás: Max eltávolítása (Második Q.remMax():)



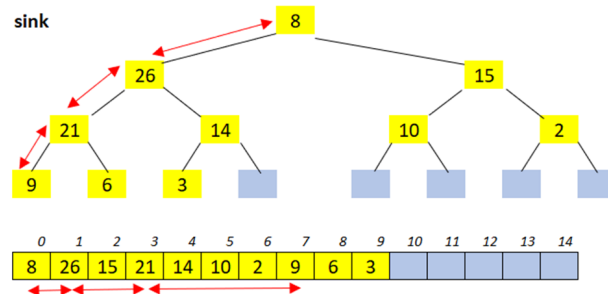
heap after second `remMax()`:



second `remMax()`:



sink



A süllyesztés (*sink*) és felszállás (*rise*) hatékonysága javítható:

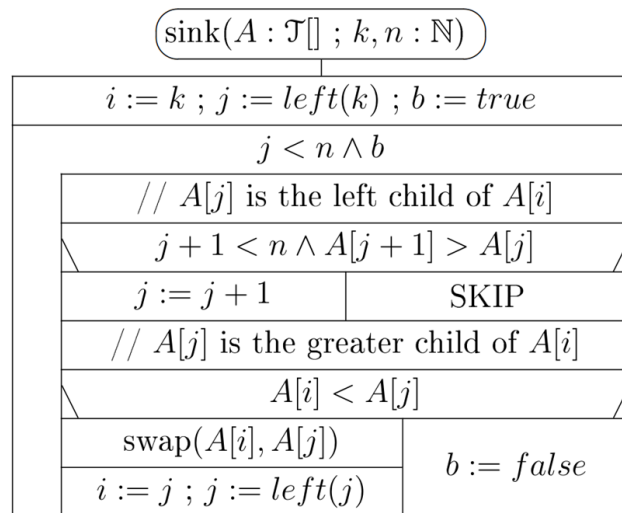
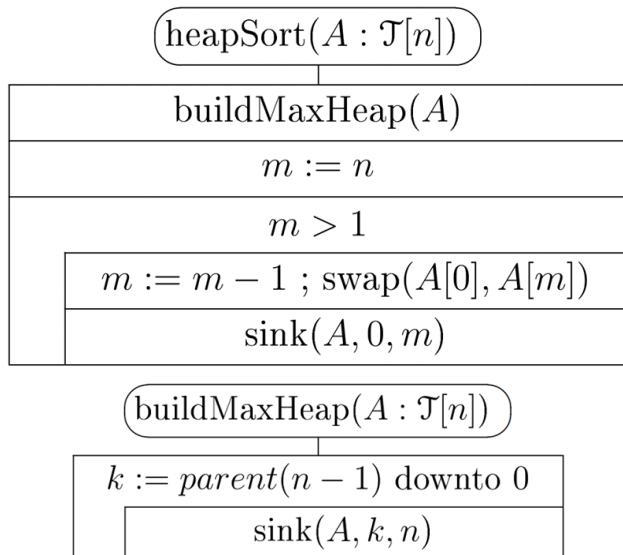
- 1 Valósítsa meg a **sink** eljárást közvetlen cserék (**swap**) nélkül!
Használjon segédváltozót a mozgatott elemnek, és csak a többi elemet másolja, amíg meg nem találja a végleges helyet.
- 2 Valósítsa meg a **rise** eljárást hasonló módon.

Kupacrendezés

Kupacrendezés (Heap sort)

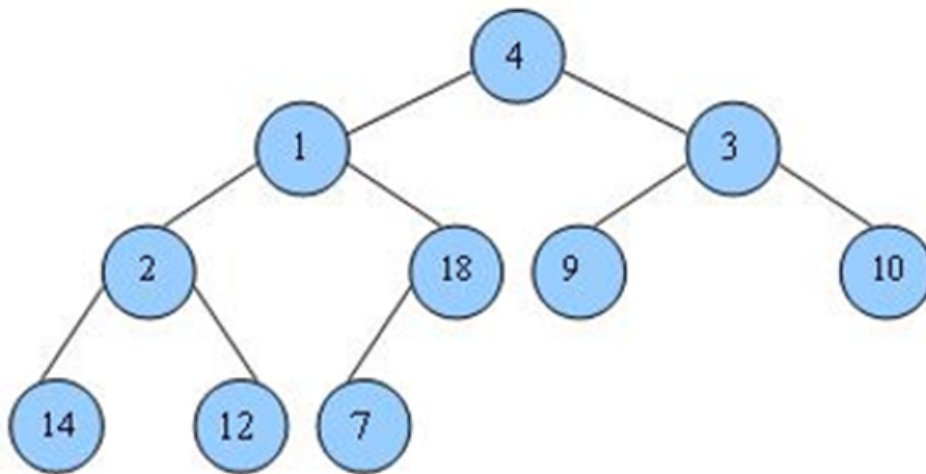
- **Időigény:** $O(n \log n)$.
- **Helyigény:** Helyben rendező (*in-place*), nem igényel plusz tárhelyet.
- **Működése:**
 - ① Kupacot építünk a tömbből (`buildMaxHeap`).
 - ② Az elemeket a helyükre mozgatjuk (n darab `remMax` süllyesztéssel).
- **Kupac építése (`buildMaxHeap`)**
 - $n/2$ darab süllyesztés (`sink`) művelet.
 - Az indexek hátulról előre haladnak: $k = \lfloor \frac{n-1}{2} \rfloor$ -től 0-ig.
 - **Bonyolultság:** $\Theta(n)$ (hatékonyabb, mintha n darab beszúrást végeznénk, ami $O(n \log n)$ lenne).

Kupacrendezés (Heap sort)



Feladat: Kupacrendezés

- Mutassuk be a kupacrendezést a következő tömbön:
- $[4, 1, 3, 2, 18, 9, 10, 14, 12, 7]$.
- A tömb a következő szintfolytonosan tárolt bináris fát tartalmazza:



Kupaccá alakítás

Step 1.: Building max heap

0	1	2	3	4	5	6	7	8	9
4	1	3	2	18	9	10	14	12	7
k					$n-1$				

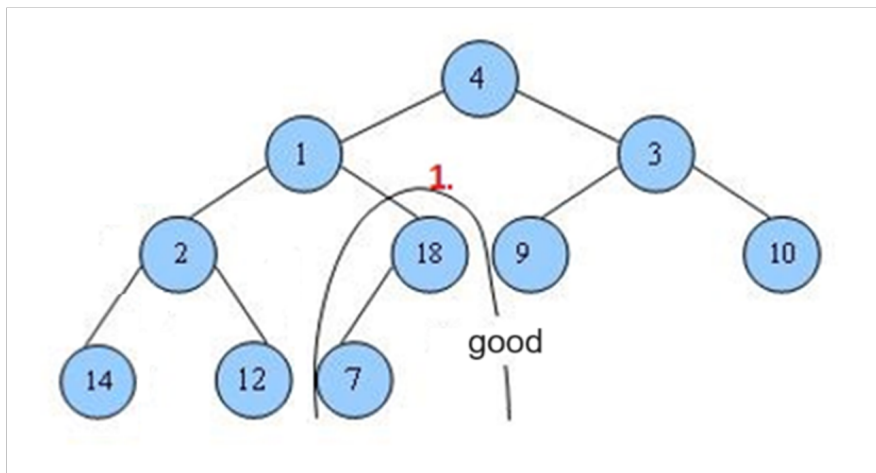
$$k := \left\lfloor \frac{n-1}{2} \right\rfloor = 4$$

1.
 $k=4$
 $\text{Sink}(A, 4, 10)$

$\text{buildMaxHeap}(A : \mathcal{T}[n])$

$k := \text{parent}(n-1)$ downto 0

$\text{sink}(A, k, n)$

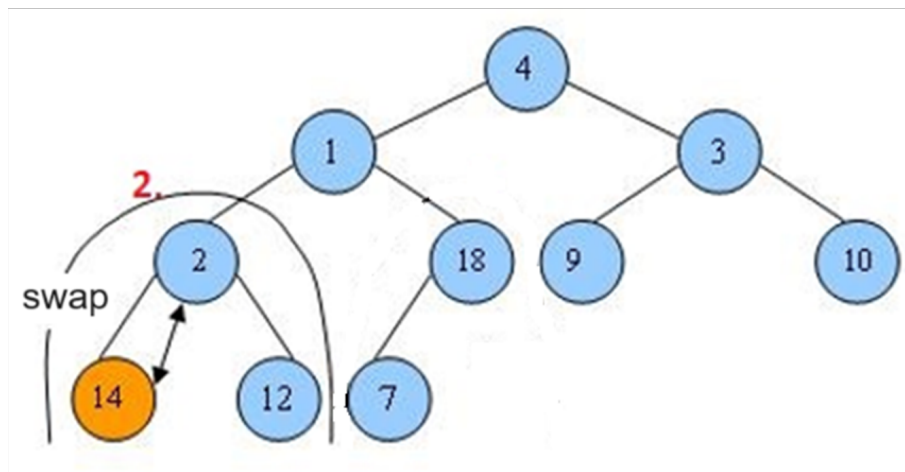
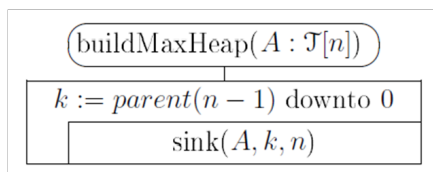


Kupaccá alakítás

Building max
heap

0	1	2	3	4	5	6	7	8	9
4	1	3	2	18	9	10	14	12	7

2.
 $k=3$
 $\text{Sink}(A, 3, 10)$

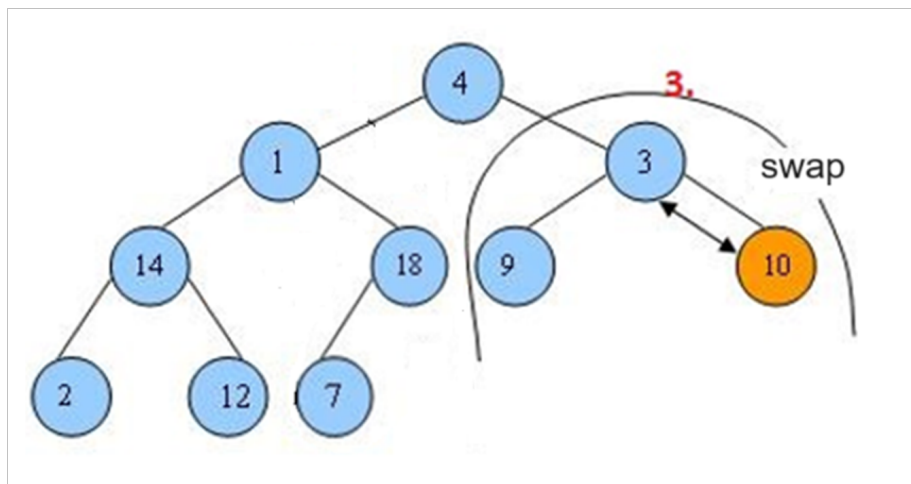
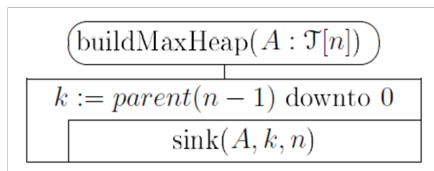


Kupaccá alakítás

Building max
heap

0	1	2	3	4	5	6	7	8	9
4	1	3	2	18	9	10	14	12	7

3.
 $k=2$
 $\text{Sink}(A, 2, 10)$



Kupaccá alakítás

Building max
heap

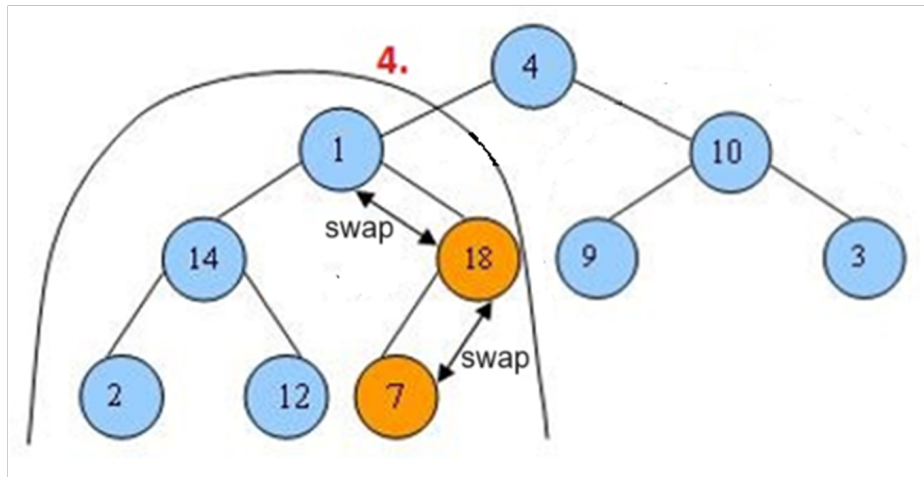
0	1	2	3	4	5	6	7	8	9
4	1	3	2	18	9	10	14	12	7

4.
 $k=1$
 $\text{Sink}(A, 1, 10)$

$\text{buildMaxHeap}(A : \mathcal{T}[n])$

$k := \text{parent}(n - 1) \text{ downto } 0$

$\text{sink}(A, k, n)$



Kupaccá alakítás

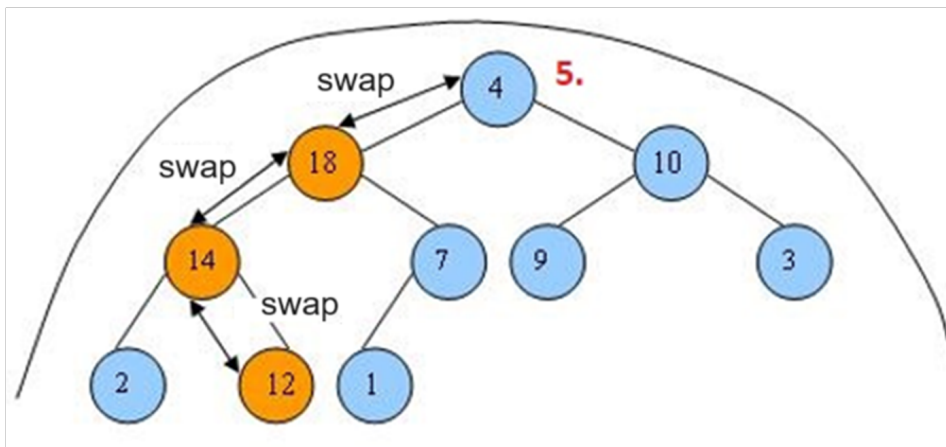
Building max heap

5.
k=0
Sink(A,0,10)

buildMaxHeap($A : \mathcal{T}[n]$)

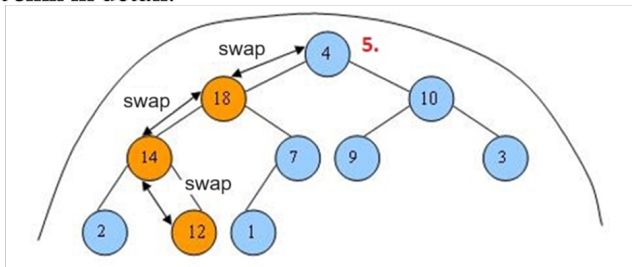
$k := \text{parent}(n - 1)$ downto 0

 sink(A, k, n)

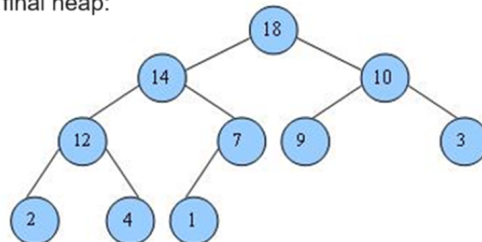


Kupaccá alakítás

Last sink in detail:

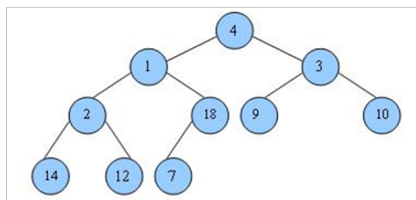


final heap:



0	1	2	3	4	5	6	7	8	9
4	18	10	14	7	9	3	2	12	1
18	4	10	14	7	9	3	2	12	1
18	14	10	4	7	9	3	2	12	1
18	14	10	12	7	9	3	2	4	1

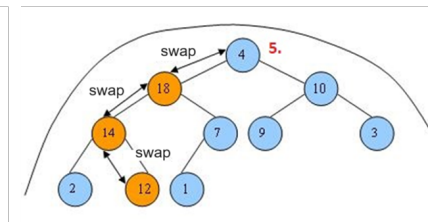
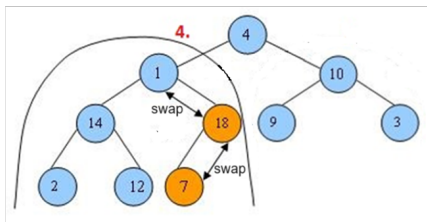
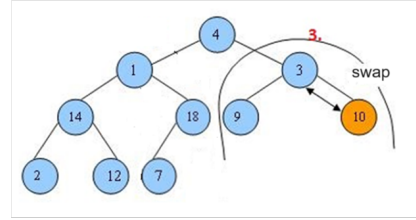
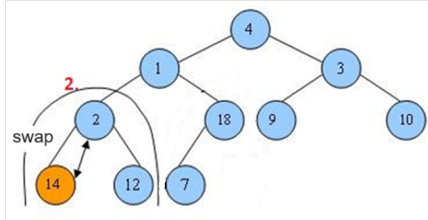
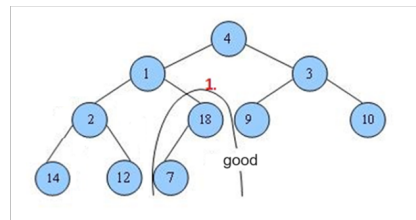
Kupaccá alakítás



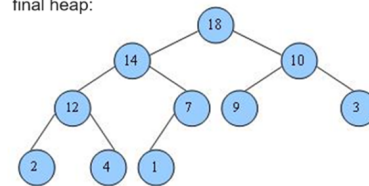
$\text{buildMaxHeap}(A : \mathcal{T}[n])$

$k := \text{parent}(n - 1) \text{ downto } 0$

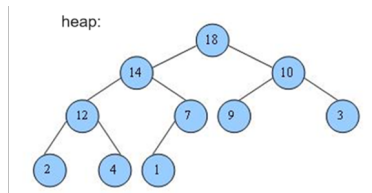
$\text{sink}(A, k, n)$



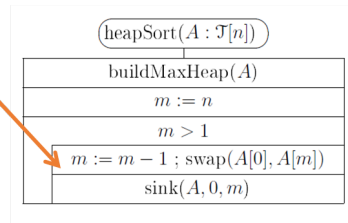
final heap:



Step 2. Sorting



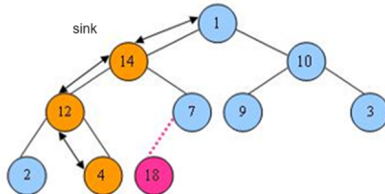
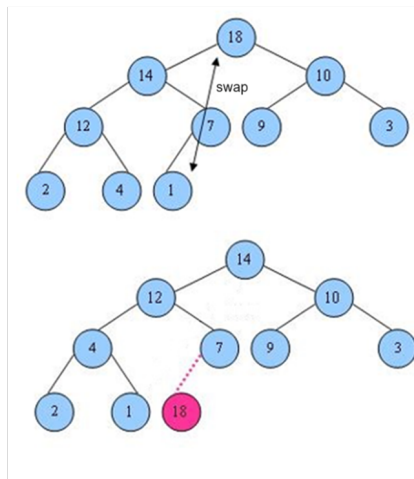
- ◆ We will only present two steps of this sorting part.



$m=10 \rightarrow m=9$

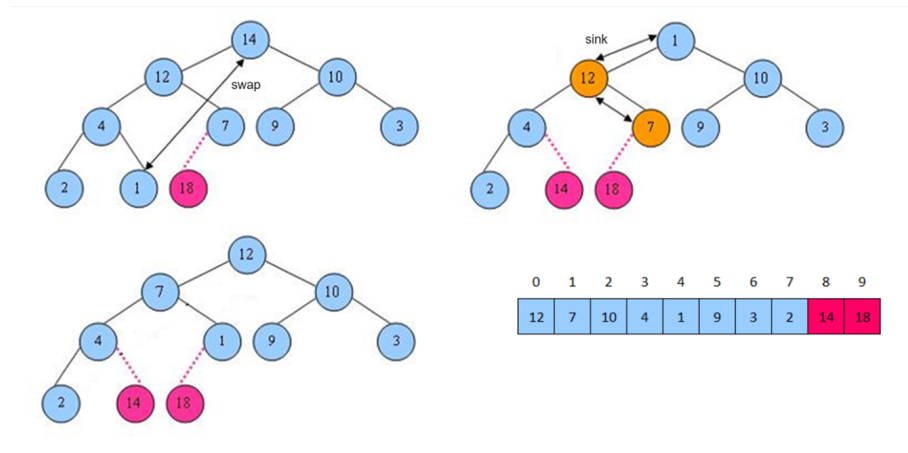
$\text{swap}(A[0], A[9])$

$\text{sink}(A, 0, 9)$

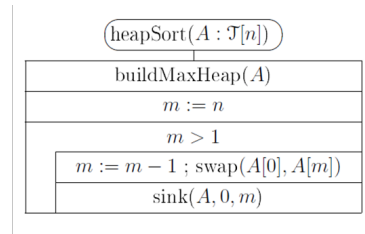


By the end of the first iteration, the heap structure is good, and the second largest element (14) is at the top. 18 is still in the array, but logically not part of the heap.

0	1	2	3	4	5	6	7	8	9
14	12	10	4	7	9	3	2	1	18



We will stop here, but the algorithm goes on until $m=1$.



$m=9 \rightarrow m=8$

$\text{swap}(A[0], A[8])$

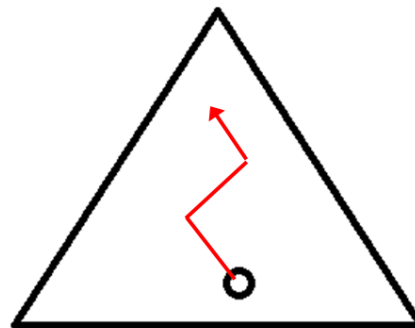
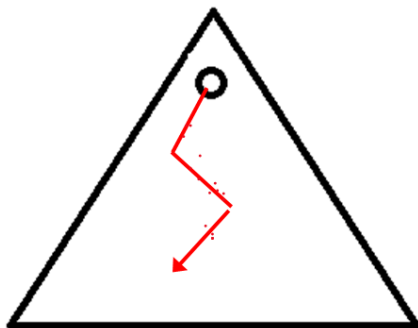
$\text{sink}(A, 0, 8)$

A „buildMaxHeap” algoritmus műveletigénye

- Könnyen igazolható, hogy a kupac kialakítása $O(n \log n)$
- Bár igazolása kissé nehezebb, de belátható a kupac kialakítása a fenti módszerrel: $\theta(n)$
- Miért nem jó megközelítés, hogyha a beszűrő művelethez hasonlóan egy üres kupacból indulnánk el és egyesével beszúrnánk az elemeket és emelés-sel a helyükre tenném őket ?

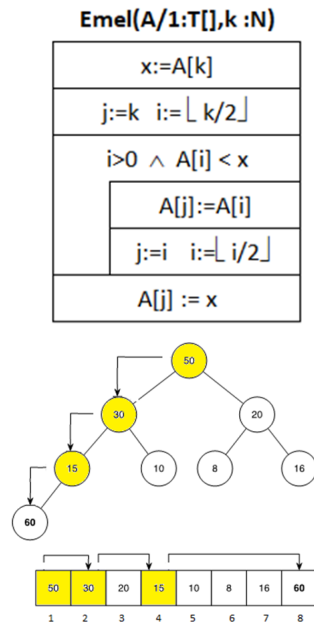
A „buildMaxHeap” algoritmus műveletigénye

- Ezzel a módszerrel a kupac kialakítása már nem lineáris műveletigényű, hanem $O(n \log n)$
- Miért?
- Az alulról felfele építkező, süllyesztéses módszer amikor egyre hosszabb a süllyesztés útvonala, már egyre kevesebb és kevesebb (mindig feleannyi elemet) süllyeszt.
- A felülről lefelé emeléses módszer amikor egyre hosszabb az emelés útvonala, mindig egyre több és több (mindig kétszer annyi) elemet emel.



Kupac hatékonnyabban

- Kupac alapl műveleteinek (emelés, süllyesztés) még hatékonyabb megvalósítása
- A bemutatott műveletek (emelés, és süllyesztés) cseréket használnak. Tudjuk, hogy minden csere három mozgatót jelent. Az algoritmusok hatékonyabbá tehetők, ha nem cseréket végeznek, hanem egy „lyukat” emelnek, vagy süllyesztenek.
- Például az emelés úgy néz ki, hogy kivesszük egy segédváltozóba az emelendő elemet, majd a lyukat visszük felfelé a kupacban, azáltal, hogy a szülőt, ha kisebb, betesszük a lyukba. Így a szülő helyére kerül a lyuk. Ezt folytatjuk, amíg kell, majd végül a lyukba betesszük a segédváltozóba kiemelt elemet. Ezzel a módszerrel a süllyesztés is hatékonyabbá tehető.



Emel(A,8)

- Készítsünk rekurzív algoritmust, mely egy láncoltan ábrázolt bináris fáról eldönti, hogy bináris keresőfa-e. A fa Node típusú elemekből áll, a kulcsok természetes számok.
- Adott egy $A[1..n]$ tömb, elemei tetszőleges típusú kulcsok, a tömb szigorúan monoton növekvő. Készítsünk rekurzív algoritmust, mely a tömb alapján felépíti az optimális alakú bináris keresőfát. A fát három pointerrel (Node3* típusú elemekkel) kell ábrázolni. A szükséges elemeket menet közben hozza létre az algoritmus, eredményül a fa gyökerére mutató pointert adjon.
- Hasonló módon készítsük el a $\text{Sink}(A[1:T]; k, n: N)$ algoritmust cserék nélkül: a süllyesztendő elemet kimásoljuk egy x segédváltozóba, majd az így keletkezett „lyukat” süllyesztjük bele a kupacba. Amikor megtaláltuk az x változóban található elem helyét, bemásoljuk a „lyukba”.

Köszönöm a figyelmet!